Diario de sesiones

Entornos de Desarrollo · 1.º DAW · Curso 2025/2026

Índice

1 Cómo usar este diario **1**
- 1.1 Reparto global por unidad . . . 2
- 1.2 El hilo conductor: «GestorTareas» . . . 2

2 Primera evaluación **3**
- 2.1 UD1 — Desarrollo del software (12 h) . . . 3
- 2.2 UD2 — Instalación y uso de entornos de desarrollo (27 h) . . . 3
- 2.3 Cierre de la 1.ª evaluación . . . 4

3 Segunda evaluación **5**
- 3.1 UD3 — Diseño y realización de pruebas (15 h) . . . 5
- 3.2 UD4 — Optimización, documentación y refactorización (14 h) . . . 5

4 Tercera evaluación **7**
- 4.1 UD5 — Elaboración de diagramas de clases (12 h) . . . 7
- 4.2 UD6 — Elaboración de diagramas de comportamiento (6 h de 12; continúa en mayo) . . . 7

5 Control de horas **8**
- 5.1 Semanas perdidas . . . 8
- 5.2 Colchón y plan B . . . 8

1 Cómo usar este diario

Este documento reparte las **89 horas lectivas** disponibles entre el inicio de curso (15/09) y el 30 de abril, respetando la temporalización de la programación del módulo y el calendario escolar de la provincia de Cádiz.

Supuestos de partida (cámbialos si tu horario real es otro):

Elemento	Valor usado
Sesión corta (1 h)	**Martes**
Sesión larga (2 h)	**Jueves**
Inicio de curso FP	15/09/2025
Fin de este diario	30/04/2026
Total horas repartidas	89 h (de las 96 h del módulo; las ~7 h restantes caen en mayo)

He elegido martes/jueves a propósito: en este calendario los días no lectivos sueltos caen en **lunes** (13/10, 08/12) y en **viernes** (10/10, 27/02), de modo que con martes y jueves no se pierde ninguna sesión fuera

de Navidad y Semana Santa. Si tu horario real cae en lunes o viernes, tendrás que absorber 2 o 3 sesiones: la columna «S01, S02…» te permite recolocar el contenido sin rehacer el reparto.

Días no lectivos aplicados: 10/10 (provincial), 13/10 (Fiesta Nacional trasladada), 01/11, 06/12, 08/12, Navidad 22/12–07/01, 27/02 (Día de la Comunidad Educativa), 28/02 (Día de Andalucía), Semana Santa 30/03–05/04, 01/05.

Ojo

El calendario oficial deja **tres días no lectivos locales** sin concretar. Cuando el centro los fije, márcalos aquí; lo más probable es que te coman 1 o 2 sesiones y tengas que recortar en UD2, que es la unidad con más colchón.

1.1 Reparto global por unidad

Ev.	UD	Título	Horas	Semanas	Fechas
1.ª	UD1	Desarrollo del software	12	S01–S04	16/09 – 09/10
1.ª	UD2	Instalación y uso de entornos de desarrollo	27	S05–S13	14/10 – 11/12
1.ª	—	Cierre y prueba de la 1.ª evaluación	3	S14	16/12 – 18/12
2.ª	UD3	Diseño y realización de pruebas	15	S17–S22	08/01 – 10/02
2.ª	UD4	Optimización, documentación y refactorización	14	S22–S26	12/02 – 12/03
3.ª	UD5	Elaboración de diagramas de clases	12	S27–S31	17/03 – 16/04
3.ª	UD6	Elaboración de diagramas de comportamiento	6 (+6 en mayo)	S32–S33	21/04 – 30/04

El orden **UD3 antes que UD4** no es el que aparece numerado en la programación heredada, pero sí es el que se siguió realmente en el centro el curso pasado, y es el defendible: refactorizar sin pruebas es exactamente lo que Fowler desaconseja. Dando primero las pruebas, la frase «puedo tocar este código porque tengo una red que me avisa si lo rompo» deja de ser un eslogan y pasa a ser algo que el alumnado comprueba en pantalla. Las dos unidades caen dentro de la **segunda evaluación**, así que el cambio no afecta a las actas ni a la ponderación por RA.

1.2 El hilo conductor: «GestorTareas»

Casi todas las prácticas giran alrededor de un mismo proyecto que crece durante el curso. Esto evita el efecto «ejercicios sueltos que no llevan a ninguna parte» y hace que el alumnado vea el módulo completo como lo que es: el ciclo de vida de una aplicación.

UD	Qué le pasa al proyecto
UD1	Se analiza y se documentan sus fases de desarrollo (todavía sin código)
UD2	Se crea el proyecto Node, se configura el entorno y se escribe la versión 0.1 (CLI)
UD3	Se le construye una batería de pruebas con Jest y se documenta el plan de pruebas
UD4	Se refactoriza (protegido por esas pruebas), se documenta con JSDoc, se pone bajo control de versiones en GitHub y se automatiza con Actions
UD5	Se modela su estructura con diagramas de clases y se genera código
UD6	Se modela su comportamiento: casos de uso, secuencia, estados y actividades

2 Primera evaluación

2.1 UD1 — Desarrollo del software (12 h)

Sem.	Fecha	Dur.	Contenido de la sesión
S01	mar 16/09	1 h	Presentación del módulo: RA, criterios, cómo se evalúa, normas del aula. Sondeo de conocimientos previos (formulario: ¿has programado?, ¿qué IDE has tocado?, ¿sabes qué es Git?).
S01	jue 18/09	2 h	1.1 Software y sistema informático. 1.2 Programa vs aplicación. 1.3 Código fuente, objeto y ejecutable. **P1.1**: abrir un `.exe`/binario con un editor hexadecimal y comparar con el `.js` y el `.c`.
S02	mar 23/09	1 h	1.4 Clasificación de lenguajes (nivel, propósito, paradigma). Puesta en común: ¿dónde encaja JavaScript?
S02	jue 25/09	2 h	1.5 Compilación vs interpretación vs JIT vs transpilación. **P1.2**: el mismo algoritmo en C, Java y JavaScript; `gcc -E -S -c`, `javac+javap`, `node`. Comparar artefactos generados.
S03	mar 30/09	1 h	1.6 Herramientas del proceso: preprocesador, compilador, enlazador, depurador, gestor de dependencias, empaquetador.
S03	jue 02/10	2 h	1.7 Ciclo de vida del software: fases. **P1.3**: dado un enunciado real («GestorTareas»), redactar en grupo qué se haría en cada fase. Entrega en Moodle.
S04	mar 07/10	1 h	1.8 Modelos de ciclo de vida I: cascada, prototipos.
S04	jue 09/10	2 h	1.8 Modelos II: incremental, espiral, ágil/Scrum. **P1.4** (comparativa con rúbrica) + **prueba escrita UD1** (40 min, tipo test + preguntas cortas).

Idea clave

La UD1 es la más teórica del módulo y la que peor tolera dos horas seguidas de pizarra. El truco está en que cada bloque teórico termine en algo que se *ve* en pantalla: el bytecode de `javap`, el `.o` de `gcc`, el árbol de un `node_modules`.

2.2 UD2 — Instalación y uso de entornos de desarrollo (27 h)

Sem.	Fecha	Dur.	Contenido de la sesión
S05	mar 14/10	1 h	2.1 Qué es un IDE y qué componentes tiene. Editor vs IDE.
S05	jue 16/10	2 h	2.2 Panorama de IDE: VS Code, IntelliJ, Eclipse, NetBeans, Visual Studio, IDE en la nube. **P2.1**: instalación de VS Code en Windows. Primer recorrido por la interfaz.
S06	mar 21/10	1 h	2.3 Software libre vs comercial: licencias (GPL, MIT, EULA), coste, soporte.
S06	jue 23/10	2 h	**P2.2**: comparativa guiada de 3 IDE con rúbrica (instalación, consumo, autocompletado, depurador, VCS). Entrega individual.
S07	mar 28/10	1 h	2.4 Configuración de VS Code: `settings.json`, perfiles, temas, atajos.
S07	jue 30/10	2 h	2.5 Extensiones: catálogo básico del módulo. **P2.3**: montar el «perfil ENDES» y exportarlo. Snippets propios.
S08	mar 04/11	1 h	2.6 Node.js: qué es, para qué lo usamos aquí. Instalación y `nvm`.
S08	jue 06/11	2 h	2.7 npm: `npm init`, `package.json`, dependencias vs devDependencies, semver. **P2.4**: crear el proyecto `gestor-tareas` desde cero.
S09	mar 11/11	1 h	2.8 Scripts de npm y automatización de tareas.
S09	jue 13/11	2 h	2.9 `tasks.json` y terminal integrada. **P2.5**: automatizar tres tareas del proyecto (arrancar, limpiar, formatear).

Sem.	Fecha	Dur.	Contenido de la sesión
S10	mar 18/11	1 h	2.10 Estructura de un proyecto: carpetas, módulos, `.editorconfig`, `.gitignore`, `.env`.
S10	jue 20/11	2 h	**P2.6**: primera versión de GestorTareas (CLI que añade, lista y borra tareas en un JSON). Trabajo guiado.
S11	mar 25/11	1 h	2.11 Entornos reproducibles: máquinas virtuales y contenedores (visión general).
S11	jue 27/11	2 h	**P2.7**: instalar VS Code + Node en Ubuntu dentro de VirtualBox y ejecutar allí el proyecto.
S12	mar 02/12	1 h	2.12 Herramientas CASE: concepto, CASE superior e inferior, ejemplos.
S12	jue 04/12	2 h	**P2.8**: primer diagrama con draw.io y con PlantUML dentro de VS Code (adelanta UD5).
S13	mar 09/12	1 h	2.13 Seguridad y buenas prácticas en el puesto de desarrollo: secretos, permisos, copias, ergonomía.
S13	jue 11/12	2 h	**P2.9** (entrega evaluable): dossier del entorno de trabajo + demo de GestorTareas v0.1 funcionando en las dos máquinas.

2.3 Cierre de la 1.ª evaluación

Sem.	Fecha	Dur.	Contenido de la sesión
S14	mar 16/12	1 h	Repaso UD1+UD2. Resolución de dudas y kahoot de repaso.
S14	jue 18/12	2 h	**Prueba de la 1.ª evaluación**: parte teórica (test) + parte práctica en el ordenador (configurar un entorno y crear un proyecto desde cero contra reloj).

Semanas S15 y S16 (22/12 – 07/01): vacaciones de Navidad.

3 Segunda evaluación

3.1 UD3 — Diseño y realización de pruebas (15 h)

Sem.	Fecha	Dur.	Contenido de la sesión
S17	jue 08/01	2 h	Entrega de notas de la 1.ª evaluación. 3.1 Por qué se prueba: coste del error, casos reales. Verificación vs validación. 3.2 Tipos de pruebas: caja blanca/negra; unitarias, integración, sistema, aceptación, regresión, carga. **P3.1**: clasificar 15 situaciones.
S18	mar 13/01	1 h	3.3 Diseño de casos de prueba: clases de equivalencia y valores límite.
S18	jue 15/01	2 h	3.4 Tablas de decisión y cobertura (sentencias, ramas, caminos). **P3.2**: diseñar la batería de casos de una función validadora de DNI.
S19	mar 20/01	1 h	3.5 Documentar las pruebas: plantilla de caso de prueba, plan de pruebas, informe de incidencia.
S19	jue 22/01	2 h	3.6 Depuración: puntos de ruptura, ejecución paso a paso, pila de llamadas, inspección, *logpoints*. **P3.3**: cazar tres errores con el depurador de VS Code (`launch.json`).
S20	mar 27/01	1 h	3.7 Pruebas unitarias automáticas: qué es Jest y cómo se instala.
S20	jue 29/01	2 h	3.8 Anatomía de una prueba: `describe`/`test`/`expect`, *matchers*, `beforeEach`. **P3.4**: primeras pruebas sobre GestorTareas.
S21	mar 03/02	1 h	3.9 TDD: ciclo rojo–verde–refactor.
S21	jue 05/02	2 h	3.10 Cobertura con `--coverage`: qué mide y qué no. 3.11 Calidad del software: ISO/IEC 25010 y certificaciones. **P3.5**: subir la cobertura de GestorTareas por encima del 80 %.
S22	mar 10/02	1 h	Cierre de la unidad y resolución de dudas. **Entrega evaluable UD3**: plan de pruebas documentado + batería Jest con ≥80 % de cobertura.

Idea clave

La integración continua (GitHub Actions) ya **no** se da aquí, sino al final de UD4, justo después de Git: montar un *workflow* antes de tener repositorio obliga a explicar Git de pasada y mal. La batería de pruebas que se entrega en esta unidad es precisamente la que se pondrá a correr en cada *push* en la sesión S26-jueves.

3.2 UD4 — Optimización, documentación y refactorización (14 h)

Sem.	Fecha	Dur.	Contenido de la sesión
S22	jue 12/02	2 h	4.1 Calidad del código y deuda técnica. 4.2 *Code smells*: catálogo con ejemplos reales.
S23	mar 17/02	1 h	4.3 Refactorización con red de seguridad: qué es, qué no es, y por qué la batería de pruebas de UD3 es el requisito previo, no un adorno.
S23	jue 19/02	2 h	4.4 Catálogo de patrones (extraer función, renombrar, eliminar duplicado, sustituir número mágico, descomponer condicional). 4.5 Refactorización asistida por el IDE: F2, *Extract function*, *Quick Fix*. **P4.1**: refactorizar un fichero deliberadamente feo **sin romper ni una sola prueba**.
S24	mar 24/02	1 h	4.6 Análisis estático: qué detecta y qué no. Métricas (complejidad ciclomática).

Sem.	Fecha	Dur.	Contenido de la sesión
S24	jue 26/02	2 h	**P4.2**: configurar ESLint + Prettier en GestorTareas y dejar el proyecto sin avisos. 4.7 Documentar el código: comentarios útiles vs ruido. JSDoc.
S25	mar 03/03	1 h	4.8 Generación automática de documentación y README.
S25	jue 05/03	2 h	**P4.3**: documentar GestorTareas y generar la web de documentación. 4.9 Control de versiones: conceptos, centralizado vs distribuido.
S26	mar 10/03	1 h	4.10 Git en la práctica (I): `init/add/commit/log/diff`, `.gitignore`, deshacer. **P4.4**: 10 ejercicios de Git.
S26	jue 12/03	2 h	4.10 Git (II): `branch/merge`, remoto y GitHub. 4.11 Integración continua con GitHub Actions ejecutando la batería de UD3 en cada *push*. **P4.5**: conflicto de *merge* provocado y resuelto por parejas. **Entrega evaluable UD4**: repositorio en GitHub con historial limpio, documentación generada, *workflow* verde y *pull request* revisada por un compañero.

Ojo

La sesión S26-jueves va muy cargada. Si el grupo va justo, deja la *pull request* revisada como tarea de casa y usa el tiempo de clase en el conflicto de *merge*, que es lo que de verdad necesitan ver con el profesor delante.

Nota: 27/02 (Día de la Comunidad Educativa) y 28/02 (Día de Andalucía) no afectan a las sesiones de martes y jueves.

4 Tercera evaluación

4.1 UD5 — Elaboración de diagramas de clases (12 h)

Sem.	Fecha	Dur.	Contenido de la sesión
S27	mar 17/03	1 h	Entrega de notas de la 2.ª evaluación. 5.1 Repaso exprés de POO: clase, objeto, atributo, método.
S27	jue 19/03	2 h	5.2 Encapsulación, herencia, polimorfismo y abstracción con ejemplos en código. 5.3 Qué es UML y qué diagramas tiene.
S28	mar 24/03	1 h	5.4 El diagrama de clases: notación, visibilidad, atributos y métodos, estáticos, abstractas e interfaces.
S28	jue 26/03	2 h	5.5 Relaciones: asociación, multiplicidad, agregación, composición, generalización, dependencia y realización. **P5.1**: leer e interpretar tres diagramas dados.
S30	mar 07/04	1 h	5.6 De los requisitos al diagrama: método de los sustantivos, responsabilidades.
S30	jue 09/04	2 h	5.7 Herramientas: PlantUML en VS Code, draw.io, StarUML. **P5.2**: modelar el caso «Biblioteca» de cero.
S31	mar 14/04	1 h	5.8 Generación de código a partir del diagrama e ingeniería inversa.
S31	jue 16/04	2 h	**P5.3 (entrega evaluable)**: diagrama de clases completo de GestorTareas + código generado + ingeniería inversa de un proyecto ajeno.

Semana S29 (30/03 – 05/04): vacaciones de Semana Santa.

4.2 UD6 — Elaboración de diagramas de comportamiento (6 h de 12; continúa en mayo)

Sem.	Fecha	Dur.	Contenido de la sesión
S32	mar 21/04	1 h	6.1 Diagramas de comportamiento: panorama y para qué sirve cada uno.
S32	jue 23/04	2 h	6.2 Casos de uso: actores, relaciones `include`/`extend`, generalización. Descripción textual de un caso de uso. **P6.1**: casos de uso de GestorTareas.
S33	mar 28/04	1 h	6.3 Diagramas de secuencia: líneas de vida, mensajes, activación.
S33	jue 30/04	2 h	6.4 Fragmentos combinados (`alt`, `opt`, `loop`) y diagrama de comunicación. **P6.2**: secuencia de «añadir tarea».

Pendiente para mayo (≈6 h): diagramas de estados, diagramas de actividad con calles, coherencia entre diagramas y proyecto integrador final. Coincide con el arranque de la fase de FP Dual que prevé la programación.

5 Control de horas

Concepto	Horas
Disponibles 16/09 – 30/04 (29 semanas lectivas × 3 h)	87
Ajuste por semana S17 (solo jueves, 2 h)	+2
Total repartido	**89**
UD1	12
UD2	27
Cierre 1.ª evaluación	3
UD3	15
UD4	14
UD5	12
UD6 (parcial)	6
Pendiente para mayo	7
Total del módulo	**96**

5.1 Semanas perdidas

Semana	Motivo
S15 (22–26 dic)	Vacaciones de Navidad
S16 (29 dic – 2 ene)	Vacaciones de Navidad
S17 (martes 06/01)	Reyes — solo hay sesión el jueves
S29 (30 mar – 3 abr)	Semana Santa

5.2 Colchón y plan B

Si te quedas sin margen —y te vas a quedar, siempre pasa— estos son los recortes por orden de menor daño:

1. **S11-jueves (P2.7, Ubuntu en VirtualBox)**: se puede convertir en demostración del profesor en 20 minutos en lugar de práctica de 2 h. Ahorro: ~1,5 h.
2. **S06-jueves (comparativa de IDE)**: pasa a trabajo de casa. Ahorro: 2 h.
3. **UD2 §2.11 (contenedores)**: es contenido de ampliación, no hay criterio de evaluación que lo exija literalmente. Ahorro: 1 h.
4. **UD5 §5.8 (ingeniería inversa)**: no se puede recortar, está en el criterio RA5-f. No toques esto.

Y al revés, si te sobra tiempo (grupo rápido), la ampliación natural es introducir TypeScript en UD5 para que la generación de código desde el diagrama de clases tenga tipos de verdad.